Propagating Developer Intent through Agentic Coordinated Refactoring

***Abstract*—Many software maintenance tasks require propagating a developer's high-level intent across numerous related program elements. While recent AI agents automate software engineering tasks end-to-end, they often struggle to consistently propagate conceptual changes throughout an entire codebase. Existing approaches either rely on program analysis, which misses semantic relationships, or on large language models (LLMs), which produce incomplete or overly broad edits.**

We introduce a novel agentic workflow for *propagating developer intent* through coordinated software refactoring. Our key idea is to represent a developer's initial change as an explicit semantic specification, called a *Declared Scope*, that captures the intended conceptual transformation and coordinates subsequent reasoning, execution, and propagation. Our workflow combines human guidance, specialized agents, and trusted IDE refactoring APIs to iteratively refine this specification and safely propagate coordinated changes across the codebase.

To understand the challenges of coordinated intent propagation, we analyzed 609K commits from 100 open-source projects and surveyed 30 developers. Motivated by these findings, we instantiated our workflow in CoRenameAgent for coordinated renaming, a representative coordinated editing task. We evaluated the approach on an established benchmark of 1349 renames and a new uncontaminated benchmark containing 1573 recent renames. CoRenameAgent substantially improves F1-score over state-of-the-art static-analysis, coding-agent, and LLM-based approaches while executing behavior-preserving refactorings through trusted IDE APIs. In a week-long user study, developers accepted 91% of the agent's suggestions, applied 1306 coordinated renames, and reported saving an average of 30 minutes per refactoring session.

I. Introduction

Software maintenance often requires developers to propagate high-level intent across many related program elements. A developer may introduce a new abstraction, rename a domain concept, migrate to a new API, or revise a design convention. Each local edit represents a broader *conceptual change* that must be propagated *consistently* across the codebase. Although such changes may appear simple, carrying them out requires developers to identify related elements, interpret project-specific conventions, and apply behavior-preserving transformations across many files. This is tedious and error-prone. Existing IDE refactoring tools update only a selected program element and its references, but do not propagate the broader developer intent behind the change.

This paper studies *coordinated renaming* as a representative coordinated editing task that requires developer intent propagation. Coordinated renaming occurs when changing one identifier requires consistently renaming multiple semantically related identifiers to preserve conceptual consistency For example, renaming the field `CustomerAccount` to `ClientAccount` may also require renaming the method `processCustomerProfile` to `processClientProfile`. More generally, coordinated editing tasks arise whenever a local edit expresses a broader conceptual change whose implications extend beyond syntactic references or program dependencies. Successfully automating these tasks therefore requires reasoning about developer intent and propagating it consistently across related program elements.

To better understand how developers propagate intent across codebases, we conducted a large-scale formative study of 609K commits across 100 open-source projects and surveyed 30 developers. Our analysis shows coordinated renaming is common (57% of rename commits), requires substantial manual effort (34 minutes on average), and typically spans multiple refactorings across several files. Some real-world commits [1] required up to 919 coordinated renames, illustrating the scale of the problem.

These findings revealed three challenges for automation:

- (1) *inferring and refining the intended scope* of a conceptual change,
- (2) *safely executing* coordinated refactorings, and
- (3) *propagating them consistently* throughout the whole codebase.

To solve these challenges, we designed CoRenameAgent, an agentic workflow for propagating developer intent through coordinated refactoring. A developer initiates a seed rename in the IDE, which serves as a signal of the intended conceptual change. CoRenameAgent represents this intent as an explicit **Declared Scope**, iteratively refines it through lightweight developer feedback, and safely propagates coordinated refactorings across the project using trusted IDE refactoring tools. Rather than searching for, remembering, and executing every related refactoring manually, developers supervise the propagation process by reviewing suggestions and refining the inferred scope. This collaboration combines human judgment, agentic reasoning, and IDE-backed execution to substantially reduce manual effort while preserving developer control, thereby addressing a key barrier to adopting AI agents—the fear of losing control and ownership [2], [3].

Existing approaches to propagating developer intent primarily follow two directions. The first one uses program analysis, which tends to be overly strict in determining the scope of renaming. Advanced IDEs [4] use heuristics, e.g., renaming a class may suggest renaming test cases. Others [5], [6], [7] follow a change-impact-analysis workflow to expand the initial renaming and apply edits accordingly. They combine static analysis (e.g., dependency relations) with NLP-based

heuristics (e.g., vocabulary similarity or structural proximity) to infer additional renames. However, as their reasoning is largely constrained by static dependencies or shallow lexical signals, they often ***fail to capture conceptual relevance among identifiers***, missing semantically related locations that should be co-renamed. Consequently, these approaches propagate program dependencies rather than developer intent.

The second direction relies on LLMs and coding agents to infer conceptual relationships beyond static program structure. LLMs can identify semantically related identifiers within local contexts, but their reasoning degrades as prompts grow to encompass large codebases [8], [9]. Recent coding agents (e.g., Codex [10], ClaudeCode [11]) overcome this limitation by autonomously navigating repositories and exploring additional files. However, their exploration is largely driven by textual similarity and heuristic search, causing them to aggressively propagate renames beyond the developer's intended conceptual change. Consequently, these agents often discover many plausible rename locations, but lack a mechanism for maintaining alignment with the developer's intent throughout project-wide exploration.

CORENAMEAGENT combines the strengths of both directions while addressing their limitations. Like LLM-based agents, it reasons about conceptual relationships beyond static dependencies. Like program-analysis approaches, it executes only behavior-preserving refactorings through trusted IDE tools. Unlike either approach, however, CORENAMEAGENT continuously aligns project-wide exploration with developer intent through an explicit **Declared Scope** that is iteratively refined using developer feedback.

The **Scope Inference Agent** begins by analysing the *seed refactoring* (i.e., a refactoring performed in the IDE or in a commit message) and transforms it into an explicit **Declared Scope**. The tool then presents renaming suggestions to the developer and collects simple go/no-go feedback, allowing **Scope Inference Agent** to refine the **Declared Scope** to align with the developer's intent (§III-A). Without developer supervision, semantic drift accumulates and propagates throughout the repository.

To address the safe execution challenge, our **Planned Execution Agent** reasons over the **Declared Scope** within each file to identify program elements that require renaming. It presents candidates to the developer for go/no-go review and executes approved renames by invoking the IDE's trusted refactoring tools, ensuring changes are safe by construction (see §III-B). The **Replication Agent** handles safe propagation through **Declared Scope**-driven file discovery, combining semantic search with program slicing to locate relevant files while avoiding unnecessary exploration (see §III-C). CORENAMEAGENT also includes an Episodic Memory that stores developer feedback and coordinates communication among agents. CORENAMEAGENT is implemented as an IntelliJ plugin using LangGraph [12] (agentic framework) and trusted IDE refactoring APIs.

We evaluate CORENAMEAGENT on two benchmarks comprising 2,922 coordinated renames, including our new uncontaminated benchmark of recent commits. CORENAMEAGENT improves F1-score by up to 3.1× over the previous state-of-the-art static-analysis approach, consistently outperforms state-of-the-art coding-agent frameworks (Codex [10] and OpenCode [13]), and improves up to 2× the coordinated-renaming capability of both proprietary (GPT) and open-source (Qwen) LLMs. An ablation study shows that removing developer feedback reduces precision by 4×, demonstrating the importance of human supervision. Finally, in a week-long study with 20 developers, participants accepted 91% of suggestions, applied 1306 coordinated renames, and reported saving 30 minutes per refactoring session.

Furthermore, we ensured that CORENAMEAGENT is practical for real developers—it runs in 5 minutes on average (compared to 1 hour for prior state-of-the-art) and reduces developer workload by performing 23 actions at an affordable cost of 30 cents. While a vanilla LLM (gpt-4o-mini) can identify some related elements to rename, it produces hallucinated edits: 45–92% of modified files fail to compile across projects, averaging 5 compilation errors per corename set. In contrast, CORENAMEAGENT identifies most elements requiring co-renaming and executes the refactorings correctly 100% of the time, with no compilation or semantic errors.

This paper makes the following contributions:

- **Agentic workflow for propagating developer intent** through coordinated refactoring, centered around the notion of a **Declared Scope** that explicitly represents and incrementally refines the intended conceptual change.
- **Case-study implementation.** We instantiate this workflow in CORENAMEAGENT, which infers the scope of co-renames and propagates them through structured reasoning and safe execution using trusted IDE refactoring APIs.
- **Empirical evaluation.** We evaluate CORENAMEAGENT on real-world Java projects, demonstrating substantial improvements over state-of-the-art static-analysis, coding-agent, and LLM-based approaches.
- **Benchmark.** We release *CoRenameBench*, a new benchmark of post-2025 coordinated renames that enables uncontaminated evaluation of LLM-based refactoring tools. The benchmark and all replication artifacts are publicly available [14].

II. MOTIVATION

Formative Study. To understand the challenges of developers performing *coordinated renaming*, we analysed the top 100 active Java projects by commit frequency. We ran RefactoringMiner [16] on all commits up to September 2025 and analysed commits containing more than two *coordinated renames*. Our analysis of 609K commits revealed 249K renames, with 57% co-occurring. On average, each *coordinated renaming* cluster contained 5 renames, changed 4 files, and updated 150 LOC. Some clusters were much larger: e.g., developers performed 919 renames [1] in a single intellij-community commit.

Following the firehouse survey method [17], [18], we monitored commit activity hourly and contacted developers who had recently performed more than five *coordinated renames*. We surveyed them about time spent, perceived difficulty,

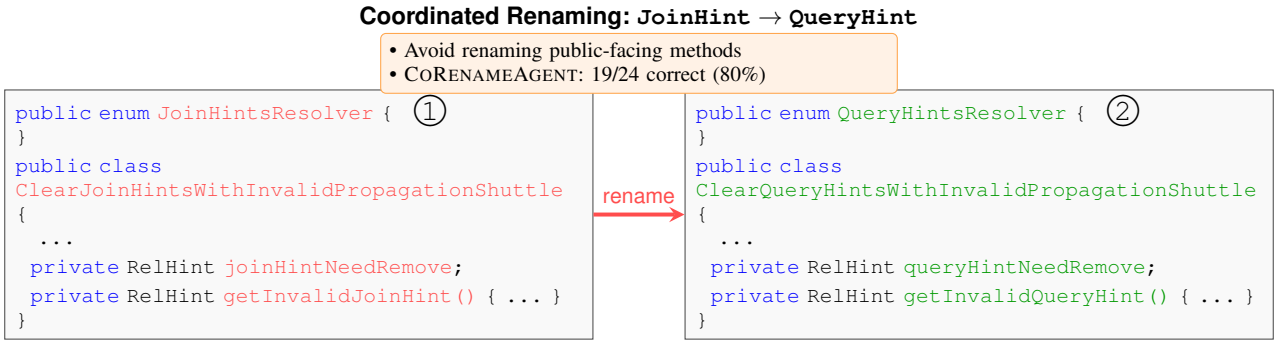


Fig. 1: Real-world example from Apache Flink commit 21403e31 [15] illustrating a *coordinated renaming*.

and automation utility. We received 30 responses from 205 contacted developers, who reported spending an average of 34 minutes on these refactorings, ranging from a few minutes to several weeks. Most respondents (16/30, 53%) described the task as tedious and error-prone, citing cross-file tracking, dependency identification, and fear of breaking changes. One developer noted: *"... my reliance on my memory ... meant that I left a few places where the old naming was still used."* All of them expressed interest in automation support.

Real World Example. Fig. 1 shows a partial view of a real-world commit [15] from Apache Flink. This involves coordinating large-scale renaming across hundreds of code sites, while also paying attention to subtle semantic differences.

In this commit, a developer performed a coordinated rename from the `JoinHint` concept to `QueryHint` (① → ②), as attested in their PR's description [19]. This coordinated change triggered renaming of 24 distinct program elements including methods, parameters, classes, fields, and local variables. The *coordinated refactoring* updated fields like `joinHints` into `queryHints`, method `capitalizeJoinHints` to `capitalizeQueryHints`, and the class `JoinHintsResolver` to `QueryHintsResolver`, creating 23 distinct pattern changes in names. However, the developer left public-facing methods like `getAllJoinHints` unchanged. This indicates that the developer's intent was to apply the *coordinated renaming only* to internal methods and fields. Overall, this coordinated rename affected 39 files resulting in 280 code site updates.

The current workflow for such large-scale, context-sensitive refactoring is tedious and error-prone. A standard IDE "Rename" feature also falls short because it operates on one identifier at a time, requiring the developer to repeat the operation for all 23 related naming patterns. Manual editing is even more challenging. The developer must find the 50 files containing `joinHint`, inspect 585 locations, identify 24 identifiers that need renaming, and update 280 LOC. Missed renames can lead to additional code-review effort; in some cases, reviewers point out missed renames by the original author, such as PR#25192 in Apache Flink [20].

Existing Approaches. We replicate the co-rename scenario in Fig. 1 example with different approaches. We found that each approach struggled. First, using a single-prompt to perform *coordinated renaming* is not feasible, as the vanilla LLM cannot access the whole project. Changes would be incomplete and confined to a single file, leading to compile errors.

Next, we used Claude Code [11] with Claude Sonnet 4.6, a state-of-the-art SWE agent from Anthropic. Unlike a vanilla LLM, Claude Code can navigate project files. We prompted it to rename variables inspired by the seed rename (`JoinHint -> QueryHint`). Claude Code renamed 6 of the 24 elements, reaching 4 of the 39 files. We also gave Claude Code access to MCP rename refactoring tools and ran it on 10 other coordinated renames from Apache Flink. On average, it operated on 8 files and recalled 27% of the renames with 21% precision. These findings align with recent work [21] showing that current SWE agents struggle with multi-file refactorings, unlike isolated function-level edits. Lastly, we used RENAS [5], a state-of-the-art research tool based on structural proximity and lexical similarity. RENAS identified only 2 of the 24 program elements to rename.

III. CoRenameAgent

We designed a human-in-the-loop agentic workflow for coordinated code editing that combines LLM-based agents with human feedback to infer, refine, and propagate intended changes. Our framework consists of three specialized LLM-based agents—**scope**, **execution**, and **replication**—augmented with external tools (Table I). These agents address different stages of coordinated editing and communicate through message passing and shared episodic memory. Their prompts are dynamically updated using human feedback and outputs from other agents, allowing each stage to refine the previous one.

Let us follow CoRenameAgent's workflow for suggesting renames on the motivating example (Fig. 1). The workflow begins when a developer performs an initial rename operation (① in Fig. 2), called the *seed rename*. The **Scope Inference Agent** analyses this *seed refactoring* in its surrounding context, to infer a broad refactoring scope – an explicit, evolving representation of the developer's intended conceptual change, called the ***Declared Scope***: "rename `'Join'` to `'Query'`." This *Declaration* then becomes a guide for all subsequent actions. The **Planned Execution Agent** uses this information to find all matching identifiers within the current file. The Planned Execution Agent presents suggestions like `joinHint` → `queryHint` to

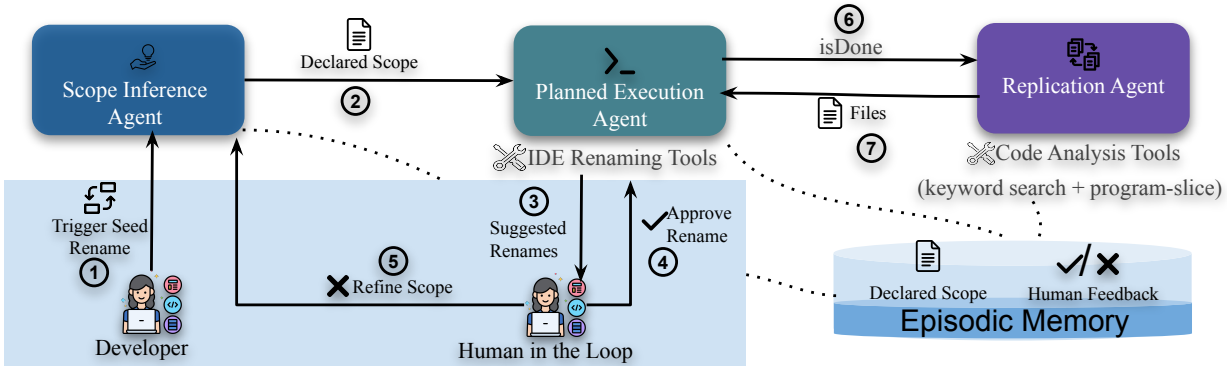


Fig. 2: Workflow of CoRenameAgent

the developer for review (③). If the developer accepts the change, CoRenameAgent triggers an IDE rename operation (④). If the developer rejects a suggestion (e.g. renaming the public method `getAllJoinHints`), CoRenameAgent invokes the Scope Inference Agent to refine the **Declared Scope** (⑤) – "avoid changes to public-facing methods". When the Planned Execution Agent completes its execution, CoRenameAgent invokes the Replication Agent to propagate changes (⑥). To handle project-wide changes, the **Replication Agent** then uses the same **Declared Scope** to identify 7 other relevant files and invokes the Planned Execution Agent to perform the renames there (⑦). Finally, CoRenameAgent successfully renamed 19/24 identifiers (80%), while presenting a high rate of valid renames to the user (77%). Next, we detail the mechanisms of each agent.

A. Scope Inference Agent *(SIA)*

SIA's task is to define the refactoring scope for CoRenameAgent. This agent is our solution to the *Scoping Challenge*: inferring and refining a general, project-wide rule based on interactions with the developer. Its goal is to curate a high-level refactoring scope from the *seed rename* and repeated interaction with the developer, called the **Declared Scope**. We define the **Declared Scope** $< p, g >$ as a tuple of two parts: pattern and guard. The 'pattern' p specifies *which* identifiers are eligible to participate in a coordinated renaming. The 'guard' g specifies *conditions* under which the name should change. Both the pattern and guard are represented in natural language.

- p is a renaming pattern of the form $old \mapsto new$,
- g is a guard (applicability condition) specifying the context in which p should apply (e.g., only to private variables).

SIA is invoked in two scenarios. (i) With the *seed rename* to generate the initial **Declared Scope** (① in Figure 2). (ii) When the developer rejects suggestions, to modify the **Declared Scope** (changing the guard condition) so future proposals avoid false positives (⑤ in Figure 2). The prompts for Scope Inference Agent are dynamic, they change each time based on the feedback from the developer.

TABLE I: Agent-Level Tools for Co-Rename Pipeline.

Agent	Tool and Description
Scope Inference (III-A)	Monitor IDE Renames: Detect rename in the IDE Monitor commit actions – Pull latest commits RefactoringMiner – Detect refactorings in a commit
Planned Execution (III-B)	Rename: Parameter, Local Variable, Method, Field, Class Update Comment
Replication (III-C)	Program Slicing – Find related locations of an element Keyword Search – Finds files containing keywords

Scope Extraction from Seed Rename. The agent starts by inferring a broad and general **Declared Scope** (① in Figure 2) by treating the *seed rename* as a clue. Initially, the agent declares only the pattern for renaming (e.g., change 'join' to 'query'), leaving the guard conditions empty. This is a deliberate design choice to allow the scope to be refined after interactions with the developer. We employ a few-shot example prompting strategy to extract the pattern, for its reasoning ability as it enables a contrastive analysis.

Scope Refinement from Developer Feedback. Complex, domain specific logic or unique project conventions can be unclear to an LLM. To handle these cases and ensure the developer retains ultimate control, the agent refines the **Declared Scope** after interaction with the developer (⑤ in Figure 2). This developer-guided feedback enables CoRenameAgent be flexible to various developers' needs. We trigger the scope refinement component when the developer rejects the suggestions. Subsequently, the Scope Inference Agent modifies the guard condition in the **Declared Scope** to accommodate the developer's reviews. We achieve this by prompting the LLM with all the developers' reviews (fetched from the episodic memory), and ask for a condition where the rename pattern should *not* be applied. This is done to ensure that LLM does not hallucinate and restrict the scope too harshly, preventing suggestions in the future. Finally, the new **Declared Scope** is stored back in memory for the other agents to access. Prompts are in the replication package [14].

Walkthrough: Scope Inference Agent. In the example from Figure 1, SIA reviews the *seed rename* and yields a **Declared Scope**: 'Perform renames following this pattern: join -> query'. After the developer rejects the suggestion to rename the public method "getAllJoinHints" to "getAllQueryHints",

the SCOPE INFERENCE AGENT refines the **Declared Scope** by adding the guard condition - "Do not apply this rename when the declaration you are touching is a public method for backward compatibility reasons".

B. PLANNED EXECUTION AGENT *(PEA)*

The goal of the PLANNED EXECUTION AGENT (PEA) is to analyse a file to suggest and execute renames that align with the **Declared Scope**. The PEA addresses two challenges: (1) accurately identifying which program elements match the **Declared Scope**, and (2) ensuring that every modification preserves the program's behaviour.

Candidate Identification. The first challenge is determining which identifiers in a file match the **Declared Scope**. We design the agent around the ReAct framework [22], enabling the PLANNED EXECUTION AGENT to reason like a developer: decide what to check, invoke a tool, observe the result, and plan the next step. We further guide the agent with task-specific few-shot examples from accepted/rejected suggestions in the *Episodic Memory*. These examples reflect the developer's scope and avoid repeated mistakes. We use a reasoning model to identify renaming candidates, as such models are effective when guided by clear objectives [22]. In this step, the PLANNED EXECUTION AGENT produces renaming suggestions, including name, type, and new name.

Refactoring Execution. The second challenge is ensuring execution safety. A naive approach would be to let an LLM directly edit the code. However, a core risk with LLMs is their tendency to hallucinate, producing code edits that are buggy [23], [24], [25]. This exposes the developer to unpredictable and potentially dangerous modifications. Therefore, to mitigate this risk, our design enforces a critical safety constraint – the agent is strictly prohibited from directly editing the source code. Instead, starting with the suggested rename candidates, PLANNED EXECUTION AGENT executes changes by invoking a curated set of IDE-backed refactoring tools:

(1) Rename identifier: rename any type of identifier - method, class, local variable, field. This tool runs safety checks before executing changes. If the agent attempts an invalid rename, a descriptive error message is returned to the agent.

(2) Update comment: a find & replace tool which updates the existing comments in the source code. This is crucial because the IDE cannot propagate renames in comments. Thus, the agent invokes a find & replace tool on the comments only, as an LLM is capable of understanding the natural language content of a comment.

Validation and Adjustment. To execute changes, the agent must convert rename suggestions into valid refactoring tool calls (III-B). However, LLMs may hallucinate the element location, such as an incorrect line number, or the element type, such as labeling a field as a variable. To address this, we build a lightweight validation wrapper that makes a best-effort attempt to locate the target identifier.

Given a list of (*identifier_name*, *identifier_type*, *new_name*, *line_number*)) from the LLM, the wrapper produces valid IDE rename calls by adjusting the line number and identifier type when needed. It inspects the file, finds declared and referenced identifiers matching the suggested name, and selects the target based on type and closeness to the expected line. If no element is found, it reports an error; if multiple elements match, it chooses the closest one by line number.

Safe Execution. The rename tools provided to the agent wrap the IDE's internal APIs, leveraging its AST and type-checking system. Thus, refactorings are safe by construction: invalid operations, such as renaming a nonexistent method, fail with descriptive errors returned to the agent for re-planning. The IDE also checks semantic preconditions before applying changes, such as preventing variable shadowing or preserving method dispatch in inheritance hierarchies. To allow renames in any order, CORENAMEAGENT identifies each target by line number; because renaming does not change line positions, these references remain stable. CORENAMEAGENT applies a rename only when all preconditions are satisfied.

Human Feedback. The PLANNED EXECUTION AGENT presents validated and safe suggestions to the developer inside the IDE, allowing them to accept or reject each suggestion (see ③ in Fig. 2). We assume the developer understands the intended conceptual realignment, either because they initiated it or discussed it during design or code review, and can judge whether a suggestion is valid. The PLANNED EXECUTION AGENT stores the developer's feedback and code snippets in memory for other agents. Based on rejections, the SCOPE INFERENCE AGENT updates the scope (⑤ in Fig. 2), and CORENAMEAGENT safely executes accepted suggestions in the IDE (④ in Fig. 2).

Termination. Finally, the **Planned Execution Agent**'s loop terminates (⑥ in Figure 2) when one of the following is satisfied: (i) the agent *reasons* that there are no more necessary renames, (ii) the iteration limit has been reached, (iii) there are no new suggestions, i.e., all the suggestions in the current iteration have already been offered before, (iv) or there are three consecutive failing tool calls.

Walkthrough: PLANNED EXECUTION AGENT. In the motivating example in Figure 1, the execution agent, starts by examining `JoinHintsResolver.java`. The agent reasons that the local field `allOptionsInJoinHints` needs to be renamed to `allOptionsInQueryHints`, and the method `initJoinHints` needs to be renamed to `initQueryHints`. It proceeds to obtain approval from the developer, and then executes these rename using the IDE tools. Then, it reasons that "there are no other program elements that need to change according to the provided scope". The execution agent then stops its execution.

C. REPLICATION AGENT *(RA)*

The REPLICATION AGENT (RA) has one task: given previously edited sites (⑥ in Figure 2), discover new files to edit according to the **Declared Scope**. RA follows two steps: 1) Discovery (edited sites → new files), and 2) Filtering. Finally, RA passes discovered files to the PLANNED EXECUTION AGENT for further analysis (⑦ in Figure 2).

The central challenge for the REPLICATION AGENT is to discover a focused set of candidate files where renames should

propagate. A brute-force analysis of every file is computationally infeasible and would overwhelm downstream agents. Instead, the task decomposes into identifying files that are: (1) structurally coupled to the initial changes, and (2) conceptually related but structurally decoupled. To address these challenges, the REPLICATION AGENT employs a two-pronged strategy: (i) it uses program slicing to analyze structural dependencies, and (ii) it performs a targeted keyword search to discover semantically-related files. We detail these steps below.

Step 1.1: Discovering Structurally-coupled Files via Program Slicing. We rely on the insight that renames often follow high-level semantic changes, such as type changes. Thus, identifiers changed together are often connected through data flow. To identify structurally coupled files, we use interprocedural program slicing. The process starts from the set of renames already executed by the tool, including the original seed rename performed by the developer and the additional elements modified by the *Planned Execution Agent* in the initial file. We use these renames as a multi-point slicing criterion to discover statements coupled through data or control dependencies. For each program element, we compute both its backward slice, i.e., statements that affect it, and its forward slice, i.e., statements affected by it. The candidate files for **Declared Scope** replication are those containing at least one structurally related statement. For scalability, slicing is bounded by recursion depth and timeout.

Step 1.2: Discovering Semantic Candidates via Keyword Search. Program slicing may not capture all relevant files, especially those linked by high-level concepts or coding conventions rather than direct data flow. Our insight is that files within the same package are often changed together. To this end, REPLICATION AGENT searches for keywords (based on the history of accepted renames) within the previously modified packages and aligned test directories. For example, if the developer accepts `capitalizeJoinHints` to `capitalizeQueryHints` in the file `src/main/com/tool/File.java`, the agent uses the keyword `capitalizeJoinHints` to search within the directories `src/main/com/tool`, and `src/test/com/tool`.

Step 2: Filtering. After the above steps, the agent is left with a set of unique candidate files. To avoid the cost and noise of running the full execution process on every file, it performs a crucial filtering step. For each candidate file, the agent asks the following question: *"Given the **Declared Scope** and the candidate file, should the refactoring be replicated in this file?"*. If the answer is yes, CORENAMEAGENT invokes the PLANNED EXECUTION AGENT to analyse that file.

The PLANNED EXECUTION AGENT and REPLICATION AGENT work hand-in-hand until a fix point is reached, iteratively discovering more files to inspect, and applying rename refactorings in those files. The PLANNED EXECUTION AGENT applies renames, and the REPLICATION AGENT inspects the codebase for other relevant files. The loop is terminated when a fix-point is reached – i.e, the REPLICATION AGENT finds no new files to inspect. For practical reasons, we terminate this fix-point algorithm after three iterations.

Walkthrough: REPLICATION AGENT (RA). On the motivating example Figure 1, the RA iteratively retrieves 61 related files. After using the LLM for filtering, we are left with 26 relevant files. The *PEA* then performs 38 refactorings.

D. Episodic Memory

An episodic memory [26] is a short-lived memory that persists only during one execution of CORENAMEAGENT and is refreshed at the start of each run. It stores facts from the current workflow, including developer interactions, past actions, and information needed for inter-agent communication.

CORENAMEAGENT uses this memory to store developer feedback on suggested renames, i.e., accept or reject, and the current **Declared Scope**. The SCOPE INFERENCE AGENT uses the feedback to refine and update the **Declared Scope**. The PLANNED EXECUTION AGENT (PEA) constructs few-shot examples from past feedback to guide its reasoning and avoid repeated suggestions. It also uses memory to keep the LLM context focused and prevent prompts from growing unnecessarily across iterations. Finally, the REPLICATION AGENT uses the stored **Declared Scope** to discover related files.

Implementation. CORENAMEAGENT's implementation details can be found in our companion technical report [Anon].

IV. EMPIRICAL EVALUATION

We designed a comprehensive, multi-methodology evaluation of CORENAMEAGENT to corroborate, complement, and expand research findings: benchmark-based evaluation (with replication of real-world refactorings), ablation analysis, operational-cost measurement, and a user study. Our evaluation addresses four questions:

RQ1. Effectiveness. How accurately does CORENAMEAGENT identify coordinated renames compared with LLM-only baselines, general-purpose coding agents, and state-of-the-art program-analysis tools?
RQ2. Ablation. Which architectural components are necessary for propagating developer intent across a project?
RQ3. Operational Cost. What are the monetary, latency, token, and interaction costs of running CORENAMEAGENT?
RQ4. Usefulness. Do developers trust CORENAMEAGENT, and does supervising the workflow reduce manual effort?

TABLE II: Evaluation datasets used in our study

Benchmark	Name	# files	LOC	# Renamings	# Sets
RenasBench [5]	RatPack	831	42K	916	109
	ArgoUML	1884	175K	443	52
	Subtotal	2715	217K	1349	161
Co-renameBench	Flink	13K	1.6M	340	39
	Kafka	5.5K	920K	211	13
	Spring-Boot	7.7K	474K	83	12
	IntelliJ-Community	82K	4.9M	85	10
	Camunda	12K	1.0M	582	60
	Bytechef	4K	244K	145	15
	MekHQ	1.4K	286K	127	8
	Subtotal	125.6K	9.4M	1573	157
	Total	128K	9.6M	2771	318

Note: K = thousands, M = millions

A. Subject Systems

We evaluate CoRenameAgent on two Java datasets to assess both reproducibility (RenasBench [5]) and generalizability (our new Co-renameBench). We use the RENAS benchmark to enable direct comparison with prior work. However, because its commits predate the training cut-offs of modern LLMs, they may have been memorized during pretraining [27], threatening the validity of LLM-based evaluation.

To mitigate this threat, we created Co-renameBench, a new benchmark mined from GitHub commits after May 2025, well beyond the May 2024 training cut-off of OpenAI's `o4-mini`. We used RefactoringMiner [28] to detect rename refactorings, then two authors manually grouped developer-performed renames into *coordinated renaming* sets.

Finally, we applied three filters to focus on non-trivial coordinated renames. First, we removed overriding-method renames, which existing IDEs already automate. Second, we retained only renames spanning multiple files. Third, we retained only instances containing at least 4 renames, based on findings from our formative study where commits with fewer than 4 renames are typically independent renames. The resulting benchmarks contain 161 RenasBench instances and 157 uncontaminated Co-renameBench instances (see Table II). We ran experiments on a 16-core MacBook with 16 GB RAM.

B. Effectiveness of CoRenameAgent (RQ1)

1) Baseline Tools: We compare CoRenameAgent against three classes of approaches. First, we evaluate whether our workflow improves the coordinated-renaming capability of underlying foundation models, both proprietary (o4-mini) and open-source (Qwen3.6 35B-A3B), chosen for their favorable cost-performance tradeoff. Second, we compare against state-of-the-art coding agents, OpenCode [13] and Codex [10]. We configured OpenCode with the Eclipse JDT Language Server [29] and the same underlying models (o4-mini and Qwen3.6) to isolate the effect of the workflow rather than the LLM. We evaluated Codex with GPT-5.4-mini; its o4-mini configuration consistently failed to modify the codebase, so we omit those results. Third, we compare against RENAS [5], the previous state-of-the-art static-analysis approach, which is representative of traditional change impact analysis tools based on program-dependencies. For reference, we also report results from vanilla prompting using GPT-4o-mini, GPT-5-mini, GPT-5, o4-mini, and Qwen3.6.

2) Evaluation Metrics: We consider a predicted rename to be a true positive (TP) if it correctly identifies a program element from the gold-set as a target for renaming. A rename is considered as a false positive (FP) if an unrelated identifier is erroneously flagged, and is not in the gold-set. All renames that are part of the gold-set but were missed by a tool are considered as false negatives (FN). We measure effectiveness using standard Precision, Recall, and F1-score with respect to the benchmark's gold-set. Note that Precision measures whether the correct identifiers are selected for coordinated renaming, not whether the underlying rename transformation preserves behavior. Transformation correctness is guaranteed by the IDE refactoring engine. We compute all metrics on the entire *coordinated renaming* set (i.e., over the entire dataset).

3) Experimental Setup: We applied the same evaluation protocol to both RenasBench and Co-renameBench. Each experiment begins from a developer-initiated seed rename and evaluates how accurately each approach propagates that intent throughout the project.

Following RENAS [5], each coordinated rename set is initiated from a single seed rename chosen using the priority Class > Field > Method > Parameter > Local Variable, reflecting the rename developers are most likely to perform first. We exclude the seed rename from evaluation.

Developer supervision is evaluated only for CoRenameAgent because it is an integral part of its algorithm. To provide a fair comparison, we additionally evaluate state-of-the-art coding agents that, like CoRenameAgent, perform iterative multi-step execution, thereby separating the benefits of autonomous agentic execution from those of developer supervision.

Because CoRenameAgent relies on developer supervision, we model the developer using a binary oracle that accepts or rejects each proposed rename. The oracle never reveals additional gold-set entries or future renames, providing only the same local feedback available to a human reviewer. Consequently, repository exploration, reasoning, and intent propagation remain entirely the responsibility of the agent—the developer merely evaluates discovered suggestions rather than helping discover new ones. The oracle validates suggestions but never guides the search. This setup enables large-scale, reproducible evaluation while faithfully modeling developer supervision.

To ensure fair comparison with RENAS, beyond running the publicly available tool with default settings, we directly corresponded with RENAS's authors to validate that our setup produced the correct output. Further, we validated our setup of RENAS by reproducing the precision and recall presented in their paper. We run CoRenameAgent with LLM temperature of 0 to maximize determinism.

TABLE III: Evaluation results of CoRenameAgent compared to baselines. RENAS* ran without errors on 75 CorenameBench entries; metrics are computed only on those runs (no penalty for failures on others).

	RenasBench			Co-renameBench		
Tool	**Prec.**	**Rec.**	**F1**	**Prec.**	**Rec.**	**F1**
RENAS * [static analysis]	14%	73%	24%	10%	35.3%	15.6%
GPT-4o-mini [LLMs]	31%	17%	22%	**71%**	10%	18%
GPT-5-mini	50%	19%	27%	60%	16%	25%
GPT-5.4-mini	56%	19%	28%	51%	16%	24%
GPT-5	**53%**	17%	26%	60%	22%	33%
o4-mini	52%	19%	28%	46%	17%	25%
Qwen3.6	59%	19%	29%	59%	17%	26%
Codex w/ gpt-5.4-mini [Agents]	36%	**83%**	50%	28%	**73%**	41%
OpenCode w/ o4-mini	38%	45%	41%	10%	43%	16%
OpenCode w/ Qwen3.6	35%	67%	46%	25%	71%	37%
CoRenameAgent w/ Qwen	44%	44%	44%	42%	40%	41%
CoRenameAgent w/ o4-mini	51.5%	58%	**54.6%**	45%	53%	**48.5%**

4) Results: Table III shows that CoRenameAgent consistently achieves the highest F1-score across both benchmarks. The gains hold across three complementary classes of baselines—static analysis, coding agents, and vanilla LLMs—indicating that explicit intent propagation, rather than any individual reasoning capability, is the missing ingredient. The three baseline categories reveal complementary limitations: static analysis lacks semantic reasoning, coding agents lack explicit intent representation, and vanilla LLMs lack project-scale coordination. CoRenameAgent addresses all.

These results suggest that the central challenge is not generating rename candidates, but maintaining alignment with developer intent during repository-wide propagation. In large projects, semantically related identifiers often span packages, layers, or modules that are difficult for LLMs to connect reliably. Moreover, the intended propagation depends on project-specific conventions—for example, whether public APIs should be renamed—which cannot be inferred from code alone. By iteratively refining the **Declared Scope** through lightweight developer feedback, CoRenameAgent adapts to evolving constraints while propagating changes.

a) Boosting the co-rename capability of an LLM: Vanilla LLMs already identify semantically related identifiers within a file, leading to high precision, but they achieve low recall because they cannot navigate and reason across files in the full project. By traversing the repository, CoRenameAgent improves recall by more than 2× for both o4-mini and Qwen. More capable LLMs may reason more accurately about individual rename candidates, but coordinated refactoring additionally requires maintaining a persistent representation of developer intent throughout repository exploration. Our workflow supplies this capability independently of the underlying model. Future advances in LLM reasoning are complementary rather than substitutive: stronger models improve individual decisions, while explicit intent propagation coordinates those decisions consistently across the repository. The architectural benefit is therefore orthogonal to advances in LLMs.

b) Comparing with frontier coding-agents–Codex and OpenCode: General-purpose coding agents successfully navigate repositories, but their precision deteriorates as exploration expands beyond the initial file because they lack an explicit representation of the developer's intended conceptual change. Although OpenCode and Codex perform repository-wide exploration, they increasingly exhibit *semantic drift*, proposing semantically plausible identifiers that fall outside the intended rename scope. In one representative example, OpenCode (with o4-mini) applied a project-wide scripted replacement changing `doc` to `document`, producing over 3,700 unnecessary renames despite achieving higher recall.

This semantic drift becomes even more pronounced as exploration moves beyond the seed file. OpenCode starts with 60% precision on the initial file but drops to 7% (i.e., 8.5× lower) on secondary files when using o4-mini. Likewise, Codex (with GPT-5.4-mini) falls from 53% to 24.5% (2× lower). In contrast, CoRenameAgent maintains substantially more stable precision across files (55% → 40%) by explicitly representing the **Declared Scope** and refining it through developer feedback rather than allowing the intended conceptual change to drift during exploration.

TABLE IV: Ablation study on *Co-renameBench*. We incrementally add the replication agent, human feedback, and scope refinement to the same base model, o4-mini.

Configuration	Precision	Recall	F1
o4-mini	46%	17%	25%
o4-mini + replication	11%	64%	19%
o4-mini + replication + human feedback	20%	63%	30%
Full CoRenameAgent	47%	56%	51.5%

These results suggest that repository navigation and iterative execution alone are insufficient for coordinated refactoring. General-purpose coding agents can effectively explore repositories, but they lack a task-specific mechanism for inferring, refining, and consistently propagating developer intent. Explicit intent propagation—not simply stronger agents—is therefore the key to accurate coordinated refactoring.

c) Comparing with static-analysis approaches: Compared with RENAS [5], the state-of-the-art static-analysis approach, CoRenameAgent improves F1-score by 2.3×–3.1×. Static analysis excels at propagating renames along explicit program dependencies such as references, call graphs, or inheritance relations, but coordinated renaming is fundamentally a *conceptual* rather than *structural* task. Semantically related identifiers often span packages, architectural layers, or naming conventions without sharing direct control-flow, data-flow, or call-graph relationships.

For example, renaming `CustomerAccount` to `ClientAccount` may also require renaming `processCustomerProfile`, even though the two identifiers might not be directly connected in the program graph. Whether such propagation is appropriate depends on the developer's intended conceptual change. By explicitly representing and refining this intent through the **Declared Scope**, CoRenameAgent propagates coordinated renames beyond syntactic relationships while remaining under developer supervision.

> Neither stronger LLMs, coding agents, nor static analysis alone solve coordinated refactoring. Explicit propagation of developer intent is the missing capability.

C. Ablation Study (RQ2)

1) Experimental Setup: We designed the ablation study to evaluate whether each architectural component addresses a distinct failure mode in propagating developer intent. Starting from a vanilla LLM (o4-mini), which reasons only within the file of the seed rename, we incrementally enable the Replication Agent, human feedback, and Scope Refinement. We evaluate each configuration on a subset of Co-renameBench containing 79 coordinated renaming instances.

2) Results: Table IV reveals a clear progression. Project-wide exploration dramatically increases recall by discovering coordinated renames beyond the initial file, but also introduces

semantic drift that causes precision to collapse. Human feedback and Scope Refinement progressively restore precision by keeping propagation aligned with developer intent, ultimately yielding the highest F1-score.

The Replication Agent overcomes the vanilla LLM's inability to propagate developer intent beyond its local context. By exploring semantically related files, it propagates the inferred **Declared Scope** throughout the project, increasing recall from 17% to 64%. However, this aggressive exploration introduces *semantic drift*, proposing plausible but out-of-scope renames and reducing precision by 4×. This shows that naive repository exploration is not sufficient for *coordinated renaming*.

Human feedback corrects this semantic drift without limiting project-wide exploration. Precision doubles by preventing semantically misaligned, out-of-scope renames from propagating further, while recall remains high. However, feedback alone corrects individual suggestions without updating the inferred scope, allowing similar mistakes to recur.

Finally, Scope Refinement generalizes developer feedback by refining the **Declared Scope** with guard conditions that guide subsequent exploration, substantially improving precision. However, Scope Inference Analysis (SIA) occasionally over-constrains the scope, filtering out valid rename candidates and slightly reducing recall.

> The ablation validates our architectural decomposition: Replication enables project-wide exploration, while human supervision and Scope Refinement are essential to prevent semantic drift and keep propagation aligned with developer intent.

D. Operational Cost Analysis (RQ3)

Experimental Setup. We quantify CORENAMEAGENT 's operational overhead per refactoring session using LangSmith to track: (1) *Monetary cost* in USD, (2) *Token consumption* (input and output), and (3) *Latency* (end-to-end time). We monitored CORENAMEAGENT (setup with `OpenAI o4-mini`) while conducting experiments over our benchmarks.

Results. We find that across all refactoring tasks in our benchmarks, the average operational cost was 30 cents per task while consuming approximately ∼77K tokens. The mean end-to-end latency was 5 minutes, consisting of Scope Extraction (10 seconds), file discovery (1.5 minutes), and planning and execution (3 minutes). Despite performing structured project-wide reasoning, CORENAMEAGENT consumes 10–20× fewer tokens than general-purpose coding agents (Table V). This efficiency stems from its task-specific workflow, which narrows exploration using the inferred **Declared Scope** instead of repeatedly reasoning over a the repository.

Latency differences with RENAS can be attributed to fundamentally different approaches: RENAS exhaustively computes program dependencies and lexical similarities across the entire codebase upfront, making it suitable only for batch processing during off-hours. In contrast, CORENAMEAGENT's scope-driven, *on-demand* approach explores only the relevant portions of the codebase. With CORENAMEAGENT's 5 minutes latency, developers can delegate the heavy lifting to it and spend minimal time reviewing high-quality suggestions.

Approach	Tokens (In – Out)	Steps	Time
Codex (GPT-5.4-mini)	1.48M – 14.9K	14	2 min
OpenCode (o4-mini)	336.7K – 1.9K	19	1.5 min
OpenCode (Qwen3-6)	783.8K – 5.7K	17	2 min
RENAS	–	–	60 min
CORENAMEAGENT (o4-mini)	*74K– 3K*	*23*	5 min

TABLE V: Operational costs of various approaches.

> Explicit intent propagation not only improves accuracy but also makes repository exploration 10–20× more token-efficient than frontier coding agents.

E. Usefulness Study (RQ4)

Experimental Setup. To assess CORENAMEAGENT 's practical utility, we recruited 20 graduate students (average 1.5 years of Java experience) who used the tool for one week on their own Java projects. Participants shared anonymous telemetry capturing fine-grained usage statistics, including accepted suggestions and review time. At the end of the study, they completed a 6-point Likert-scale survey measuring perceived suggestion quality, supervision effort, and estimated time savings. The survey instrument and responses are available in our replication package [14].

Results. Over the one-week study, participants invoked CORENAMEAGENT 201 times. Participants consistently trusted its project-wide propagation, accepting 91% of the proposed coordinated renames, resulting in 1306 applied renames. Each session generated 8 suggestions on average, keeping supervision lightweight. Participants refined the inferred **Declared Scope** only 9 times across all sessions, indicating that the inferred scope correctly captured the intended change.

Developer supervision required relatively little effort. On average, CORENAMEAGENT executed for 7 minutes per session, while developers spent only 3.5 minutes reviewing suggestions. During that time, the agent analyzed 11 files and inspected 157 unique identifiers to produce just 8 coordinated rename suggestions spanning 5 files, hiding the complexity of repository-wide exploration behind a lightweight supervision interface. Participants estimated that this saved an average of 30 minutes compared to performing coordinated renaming manually. Moreover, CORENAMEAGENT was faster than manual refactoring (70% Agree or better), and the required level of supervision was appropriate (75% Agree or better).

In survey responses, participants consistently highlighted CORENAMEAGENT 's ability to *propagate conceptual changes across a project*. One participant reported that it "saved significant time by finding related terms", while another described using it to transform singular identifiers into plural ones throughout a project—a coordinated edit they found difficult to perform manually. Compared with frontier coding agents, participants noted that CORENAMEAGENT prioritized accurate, developer-supervised suggestions over speed.

Developers trusted CoRenameAgent 's project-wide propagation, accepting 91% of suggestions while spending only 3.5 minutes supervising each session, saving an average of 30 minutes per coordinated refactoring.

V. Threats to Validity

Internal Validity. To mitigate the non-determinism of LLMs, we executed each refactoring using a fixed model and temperature of 0.

External Validity. We evaluated CoRenameAgent exclusively on open-source Java projects, which may limit the generalizability of our findings to other programming languages or proprietary codebases. However, the core multi-agent framework of CoRenameAgent (i.e., separating Scope Extraction from tool-driven propagation) is language-agnostic. To adapt CoRenameAgent to another language like Python, one would need to replace the Java-specific components (rename and program-slicing implementations) while the fundamental agentic workflow remains the same.

Dataset Validity. We build our benchmarks assuming that developers perform *coordinated renaming* comprehensively in a single commit. In practice, some renames may be delayed or distributed across multiple commits, for example, when a reviewer later points out an omission. As a result, our oracle derived from commits may be incomplete, leading to false negatives (i.e., valid but unobserved renames), though false positives are unlikely given the use of mature, reviewed repositories. Consequently, suggestions not present in the oracle may be penalized despite being semantically valid. Future work could link related commits to capture a complete view of *coordinated renaming* activity, thereby expanding the oracle.

VI. Related Work

We group related work into: (1) Change Impact Analysis, (2) suggesting Rename Refactoring, and (3) LLM-based Refactoring Agents.

Change Impact Analysis. Change Impact Analysis (CIA) identifies parts of a system affected by a code change, supporting testing and maintenance [30], [31], [32], [33], [34], [35]. Existing techniques estimate impact using code structure and behavior [31], [36], [37], [38], external artifacts such as change requests and execution data [39], [40], [41], [42], and version-history patterns [43]. Other approaches combine multiple information sources for more accurate impact prediction [44], [45], [46]. Recent work further uses LLMs to capture developer intent and predict change impact [47].

CIA approaches focus on locating impacted locations, but do not suggest specific changes. CoRenameAgent can be viewed as a specialized change-impact-analysis workflow for coordinated renaming. It uses LLMs and static analysis, and leverages human feedback to suggest renames that align with the original change.

Identifier renaming is one of the most frequent refactorings in modern codebases [48], [5], [6]. Many prior techniques focus on recommending improved names for individual identifiers [48], [49], [50], [51], [52], [53], often integrated into IDEs for automated replacement. Our approach complements these existing techniques by focusing on where renaming should propagate, not just what name to choose.

Discovering *coordinated renaming*. *Coordinated renaming* focuses on identifying which identifiers should be renamed together. Empirical studies show that more than half of renames affect multiple elements [6], motivating automated support. RENAS [5] uses the call-graph to extract relational knowledge, while RenameExpander [7] uses structural proximity. CoRenameAgent is designed to capture human intent, enabling it to propagate renames even when syntactic cues are weak.

Refactoring with LLMs and Agents. LLMs/agents have been used for suggesting method-level refactorings (e.g., extract/move method) [23], [24], [54], code-smell removal [55], and identifying modularity targets [56]. These systems propose new edits, while CoRenameAgent propagates renames already initiated by the developer.

Coding agents. SWE-agents possess generalist programming capabilities [11], [57], [58], [59], [60]. Recent studies [21], [61], [62] have evaluated the capabilities of coding agents when performing complex refactoring tasks. RefactorBench [21] and CodeTaste [61] introduce benchmarks of multi-file refactoring tasks designed to assess stateful reasoning in agents. They find that SWE agents struggle with long edits, and required detailed prompts to achieve good results. Moreover, most coding agents operate in a request-and-response fashion that reduces the developers' role to validate the agent's output. However, our workflow is collaborative, and adapts continuously based on user feedback.

VII. Conclusion

Our work shows that coordinated editing tasks, often viewed as routine maintenance, are fundamentally *intent-propagation problems*. Successfully automating them requires explicitly representing the developer's intended conceptual change, refining it through lightweight supervision, and propagating it consistently across a codebase.

We instantiate this paradigm through CoRenameAgent, where the **Declared Scope** captures developer intent and is continuously refined during project-wide propagation. Rather than relying solely on stronger foundation models or unrestricted repository exploration, our workflow combines structured reasoning, trusted IDE-backed execution, and developer supervision to keep coordinated changes aligned with the intended conceptual change. A central finding is that developer supervision is essential not for discovering renames, but for refining the **Declared Scope** and preventing semantic drift, shifting developers from searching, remembering, and executing coordinated edits to supervising their propagation.

Beyond coordinated renaming, explicit intent propagation through a continuously refined **Declared Scope** provides a general blueprint for agent-assisted software engineering. We hope this work inspires the community to investigate broader classes of coordinated maintenance and software evolution tasks where developer intent can be explicitly represented, refined, and propagated through human–AI collaboration.

REFERENCES

[1] "cleanup: rename file to psifile to distinguish from virtualfile," https://github.com/JetBrains/intellij-community/commit/6d1f55f16043ad48f67762c50ab1035e0a589ca7, 2025, accessed: 2025-07-16.

[2] A. Sergeyuk, Y. Golubev, T. Bryksin, and I. Ahmed, "Using AI-based coding assistants in practice: State of affairs, perceptions, and ways forward," *Information and Software Technology*, vol. 178, p. 107610, 2025.

[3] A. Kumar, Y. Bajpai, S. Gulwani, G. Soares, and E. Murphy-Hill, "Why AI Agents Still Need You: Findings from Developer-Agent Collaborations in the Wild," in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE Press, 2025, p. 432–444. [Online]. Available: https://doi.org/10.1109/ASE63991.2025.00043

[4] JetBrains, "IntelliJ IDEA," https://www.jetbrains.com/idea/, The Leading IDE for Professional Development.

[5] N. Doi, Y. Osumi, and S. Hayashi, "RENAS: Prioritizing Co-Renaming Opportunities of Identifiers," in *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2024, pp. 562–573.

[6] Y. Osumi, N. Umekawa, H. Komata, and S. Hayashi, "Empirical Study of Co-Renamed Identifiers," in *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 2022, pp. 71–80.

[7] H. Liu, Q. Liu, Y. Liu, and Z. Wang, "Identifying Renaming Opportunities by Expanding Conducted Rename Refactorings," *IEEE Transactions on Software Engineering*, vol. 41, no. 9, pp. 887–900, 2015.

[8] "Needle In A Haystack - Pressure Testing LLMs," https://github.com/gkamradt/LLMTest_NeedleInAHaystack.

[9] F. Shi, X. Chen, K. Misra, N. Scales, D. Dohan, E. H. Chi, N. Schärli, and D. Zhou, "Large language models can be easily distracted by irrelevant context," in *International Conference on Machine Learning*. PMLR, 2023, pp. 31 210–31 227.

[10] OpenAI, "Codex | AI Coding Partner from OpenAI," 2026. [Online]. Available: https://openai.com/codex/

[11] Anthropic, "Claude Code," https://www.anthropic.com/claude-code, 2025, accessed: 2025-10-08.

[12] LangChain Community, "LangGraph," https://github.com/langchain-ai/langgraph, open-source graph-based orchestration library for LLM workflows.

[13] "OpenCode | The open source AI coding agent," 2026. [Online]. Available: https://opencode.ai/

[14] "CoRenameAgent replication package," July 2026. [Online]. Available: https://renameagent.netlify.app/

[15] "Commit - Support state ttl hint for regular join," https://github.com/apache/flink/commit/21403e31, accessed: 2025-09-29.

[16] N. Tsantalis, A. Ketkar, and D. Dig, "RefactoringMiner 2.0," *IEEE Transactions on Software Engineering*, vol. 48, no. 3, pp. 930–950, 2022.

[17] E. Murphy-Hill, T. Zimmermann, C. Bird, and N. Nagappan, "The Design Space of Bug Fixes and How Developers Navigate It," *IEEE Transactions on Software Engineering*, vol. 41, no. 1, pp. 65–81, 2015.

[18] D. Mazinanian, A. Ketkar, N. Tsantalis, and D. Dig, "Understanding the use of lambda expressions in Java," *Proc. ACM Program. Lang.*, vol. 1, no. OOPSLA, Oct. 2017. [Online]. Available: https://doi.org/10.1145/3133909

[19] "[FLINK-33583][table-planner] Support state ttl hint for regular join," https://github.com/apache/flink/pull/23752, accessed: 2025-09-29.

[20] "[FLINK-34510][Runtime/State]Rename RestoreMode to RecoveryClaimMode," https://github.com/apache/flink/pull/25192, accessed: 2025-09-29.

[21] D. Gautam, S. Garg, J. Jang, N. Sundaresan, and R. Z. Moghaddam, "RefactorBench: Evaluating Stateful Reasoning in Language Agents Through Code," in *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://openreview.net/forum?id=NiNIthntx7

[22] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *International Conference on Learning Representations (ICLR)*, 2023.

[23] D. Pomian, A. Bellur, M. Dilhara, Z. Kurbatova, E. Bogomolov, A. Sokolov, T. Bryksin, and D. Dig, "EM-Assist: Safe Automated ExtractMethod Refactoring with LLMs," in *FSE*, 2024, pp. 582–586.

[24] A. Bellur, F. Batole, M. Dilhara, M. R. Ullah, Y. Zharov, T. Bryksin, K. Ishikawa, H. Chen, M. Morimoto, S. Motoura, T. Hosomi, T. N. Nguyen, H. Rajan, N. Tsantalis, and D. Dig, "Together We Are Better: LLM, IDE and Semantic Embedding to Assist Move Method Refactoring," in *International Conference on Software Maintainance and Evolution (ICSME)*, 2025, pp. 1–13.

[25] M. Dilhara, A. Bellur, T. Bryksin, and D. Dig, "Unprecedented Code Change Automation: The Fusion of LLMs and Transformation by Example," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 631–653, 2024.

[26] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths, "Cognitive Architectures for Language Agents," 2024. [Online]. Available: https://arxiv.org/abs/2309.02427

[27] Y. Dong, X. Jiang, H. Liu, Z. Jin, B. Gu, M. Yang, and G. Li, "Generalization or memorization: Data contamination and trustworthy evaluation for large language models," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 12 039–12 050.

[28] N. Tsantalis, M. Mansouri, L. M. Eshkevari, D. Mazinanian, and D. Dig, "Accurate and Efficient Refactoring Detection in Commit History," in *ICSE*, 2018, pp. 483–494.

[29] Eclipse, "Eclipse JDT Language Server," 2026. [Online]. Available: https://github.com/eclipse-jdtls/eclipse.jdt.ls

[30] G. Rothermel and M. J. Harrold, "A Safe, Efficient Regression Test Selection Technique," *ACM Trans. Softw. Eng. Methodol.*, vol. 6, no. 2, pp. 173–210, 1997. [Online]. Available: https://doi.org/10.1145/248233.248262

[31] X. Ren, F. Shah, F. Tip, B. G. Ryder, and O. Chesley, "Chianti: a tool for change impact analysis of java programs," in *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*, ser. OOPSLA '04. New York, NY, USA: Association for Computing Machinery, 2004, p. 432–448. [Online]. Available: https://doi.org/10.1145/1028976.1029012

[32] M. Gligoric, L. Eloussi, and D. Marinov, "Practical regression test selection with dynamic file dependencies," in *Proceedings of the 2015 International Symposium on Software Testing and Analysis, (ISSTA 2015), Baltimore, MD, USA, July 12-17, 2015*. IEEE CS, 2015, pp. 211–222. [Online]. Available: https://doi.org/10.1145/2771783.2771784

[33] T. Le, T. Tran, D. Cao, V. Le, T. N. Nguyen, and V. Nguyen, "KAT: Dependency-Aware Automated API Testing with Large Language Models," in *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2024, pp. 82–92. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/ICST60714.2024.00017

[34] O. C. Chesley, X. Ren, and B. G. Ryder, "Crisp: A Debugging Tool for Java Programs," in *Proceedings of the 21st IEEE International Conference on Software Maintenance (ICSM 2005), 25-30 September 2005, Budapest, Hungary*. IEEE CS, 2005, pp. 401–410. [Online]. Available: https://doi.org/10.1109/ICSM.2005.37

[35] F. Fioravanti and P. Nesi, "Estimation and Prediction Metrics for Adaptive Maintenance Effort of Object-Oriented Systems," *IEEE Trans. Software Eng.*, vol. 27, no. 12, pp. 1062–1084, 2001. [Online]. Available: https://doi.org/10.1109/32.988708

[36] T. T. Nguyen, H. A. Nguyen, N. H. Pham, J. Al-Kofahi, and T. N. Nguyen, "Recurring bug fixes in object-oriented programs," in *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering - Volume 1*, ser. ICSE '10. New York, NY, USA: Association for Computing Machinery, 2010, p. 315–324. [Online]. Available: https://doi.org/10.1145/1806799.1806847

[37] B. Li, X. Sun, H. Leung, and S. Zhang, "A survey of code-based change impact analysis techniques," *Softw. Test. Verification Reliab.*, vol. 23, no. 8, pp. 613–646, 2013. [Online]. Available: https://doi.org/10.1002/stvr.1475

[38] D. Poshyvanyk, A. Marcus, R. Ferenc, and T. Gyimóthy, "Using information retrieval based coupling measures for impact analysis," *Empirical Softw. Engg.*, vol. 14, no. 1, p. 5–32, Feb. 2009. [Online]. Available: https://doi.org/10.1007/s10664-008-9088-2

[39] M. Shiri, J. Hassine, and J. Rilling, "Modification analysis support at the requirements level," in *Proceedings of the 9th International Workshop on Principles of Software Evolution (IWPSE 2007), in conjunction with the 6th ESEC/FSE joint meeting, Dubrovnik, Croatia, September 3-4, 2007*, 2007, pp. 43–50. [Online]. Available: https://doi.org/10.1145/1294948.1294961

[40] L. C. Briand, Y. Labiche, and L. O'Sullivan, "Impact analysis and change management of UML models," in *Proceedings of the*

19th International Conference on Software Maintenance (ICSM'03), The Architecture of Existing Systems, 22-26 September 2003, Amsterdam, The Netherlands, 2003, pp. 256–265. [Online]. Available: https://doi.org/10.1109/ICSM.2003.1235428

[41] A. Orso, T. Apiwattanapong, and M. J. Harrold, "Leveraging field data for impact analysis and regression testing," in *Proceedings of the 9th European Software Engineering Conference Held Jointly with 11th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, ser. ESEC/FSE-11. New York, NY, USA: Association for Computing Machinery, 2003, p. 128–137. [Online]. Available: https://doi.org/10.1145/940071.940089

[42] T. Apiwattanapong, A. Orso, and M. Harrold, "Efficient and precise dynamic impact analysis using execute-after sequences," in *Proceedings of the 27th International Conference on Software Engineering (ICSE 2005)*. IEEE CS, 2005, pp. 432–441.

[43] T. Zimmermann, S. Kim, A. Zeller, and E. J. Whitehead, "Mining version archives for co-changed lines," in *Proceedings of the 2006 International Workshop on Mining Software Repositories*, ser. MSR '06. New York, NY, USA: ACM, 2006, p. 72–75. [Online]. Available: https://doi.org/10.1145/1137983.1138001

[44] L. Huang and Y.-T. Song, "Precise dynamic impact analysis with dependency analysis for object-oriented programs," in *Proceedings of the 5th ACIS International Conference on Software Engineering Research, Management & Applications*, ser. SERA '07. USA: IEEE Computer Society, 2007, p. 374–384. [Online]. Available: https://doi.org/10.1109/SERA.2007.109

[45] H. Kagdi, M. Gethers, D. Poshyvanyk, and M. L. Collard, "Blending conceptual and evolutionary couplings to support change impact analysis in source code," in *Proceedings of the 2010 17th Working Conference on Reverse Engineering*, ser. WCRE '10. USA: IEEE Computer Society, 2010, p. 119–128. [Online]. Available: https://doi.org/10.1109/WCRE.2010.21

[46] Y. Yan, N. Cooper, K. Moran, G. Bavota, D. Poshyvanyk, and S. Rich, "Enhancing code understanding for impact analysis by combining transformers and program dependence graphs," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 972–995, Jul. 2024. [Online]. Available: https://doi.org/10.1145/3643770

[47] A. Yadavally and T. N. Nguyen, "From Seed to Scope: Reasoning to Identify Change Impact Sets," *Proceedings of the 58th International Conference on Software Engineering (ICSE)*, 2026.

[48] P. Jablonski and D. Hou, "CReN: A tool for tracking copy-and-paste code clones and renaming identifiers consistently in the IDE," in *Proceedings of the 2007 OOPSLA workshop on eclipse technology eXchange*, 2007, pp. 16–20.

[49] S. L. Abebe and P. Tonella, "Automated identifier completion and replacement," in *2013 17th European Conference on Software Maintenance and Reengineering*. IEEE, 2013, pp. 263–272.

[50] C. Dong, Y. Jiang, N. Niu, Y. Zhang, and H. Liu, "Context-Aware Name Recommendation for Field Renaming," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.

[51] J. Zhang, J. Luo, J. Liang, L. Gong, and Z. Huang, "An accurate identifier renaming prediction and suggestion approach," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 6, pp. 1–51, 2023.

[52] T. Wang, Y. Zhang, L. Jiang, Y. Tang, G. Li, and H. Liu, "Deep learning based identification of inconsistent method names: How far are we?" *Empirical Software Engineering*, vol. 30, no. 1, p. 31, 2025.

[53] H. Liu, Y. Wang, Z. Wei, Y. Xu, J. Wang, H. Li, and R. Ji, "RefBERT: A Two-Stage Pre-trained Framework for Automatic Rename Refactoring," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 740–752.

[54] Y. Xu, F. Lin, J. Yang, N. Tsantalis *et al.*, "MANTRA: Enhancing Automated Method-Level Refactoring with Contextual RAG and Multi-Agent LLM Collaboration," *arXiv preprint arXiv:2503.14340*, 2025.

[55] D. Wu, F. Mu, L. Shi, Z. Guo, K. Liu, W. Zhuang, Y. Zhong, and L. Zhang, "iSMELL: Assembling LLMs with Expert Toolsets for Code Smell Detection and Refactoring," in *ASE*, 2024.

[56] F. Batole, D. OBrien, T. Nguyen N., R. Dyer, and H. Rajan, "An LLM-Based Agent-Oriented Approach for Automated Code Design Issue Localization," in *2025 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, IEEE. ICSE, 2025.

[57] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zheng, B. Qian, Y. Shao, N. Muennighoff, Y. Zhang, B. Hui, J. Lin, R. Brennan, H. Peng, H. Ji, and G. Neubig, "OpenHands: An Open Platform for AI Software Developers as Generalist Agents," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=OJd3ayDDoF

[58] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. R. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. [Online]. Available: https://arxiv.org/abs/2405.15793

[59] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "Autocoderover: Autonomous program improvement," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1592–1604.

[60] Amazon Web Services, Inc., "Amazon Q Developer," https://aws.amazon.com/q/developer/, 2026.

[61] A. Thillen, N. Mündler, V. Raychev, and M. Vechev, "CodeTaste: Can LLMs Generate Human-Level Code Refactorings?" Mar. 2026, arXiv:2603.04177 [cs]. [Online]. Available: http://arxiv.org/abs/2603.04177

[62] J. Cordeiro, S. Noei, and Y. Zou, "An Empirical Study on the Code Refactoring Capability of Large Language Models," *ACM Trans. Softw. Eng. Methodol.*, p. 3801158, Mar. 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3801158